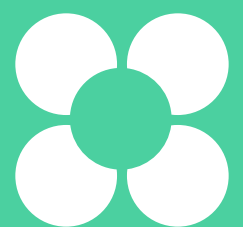
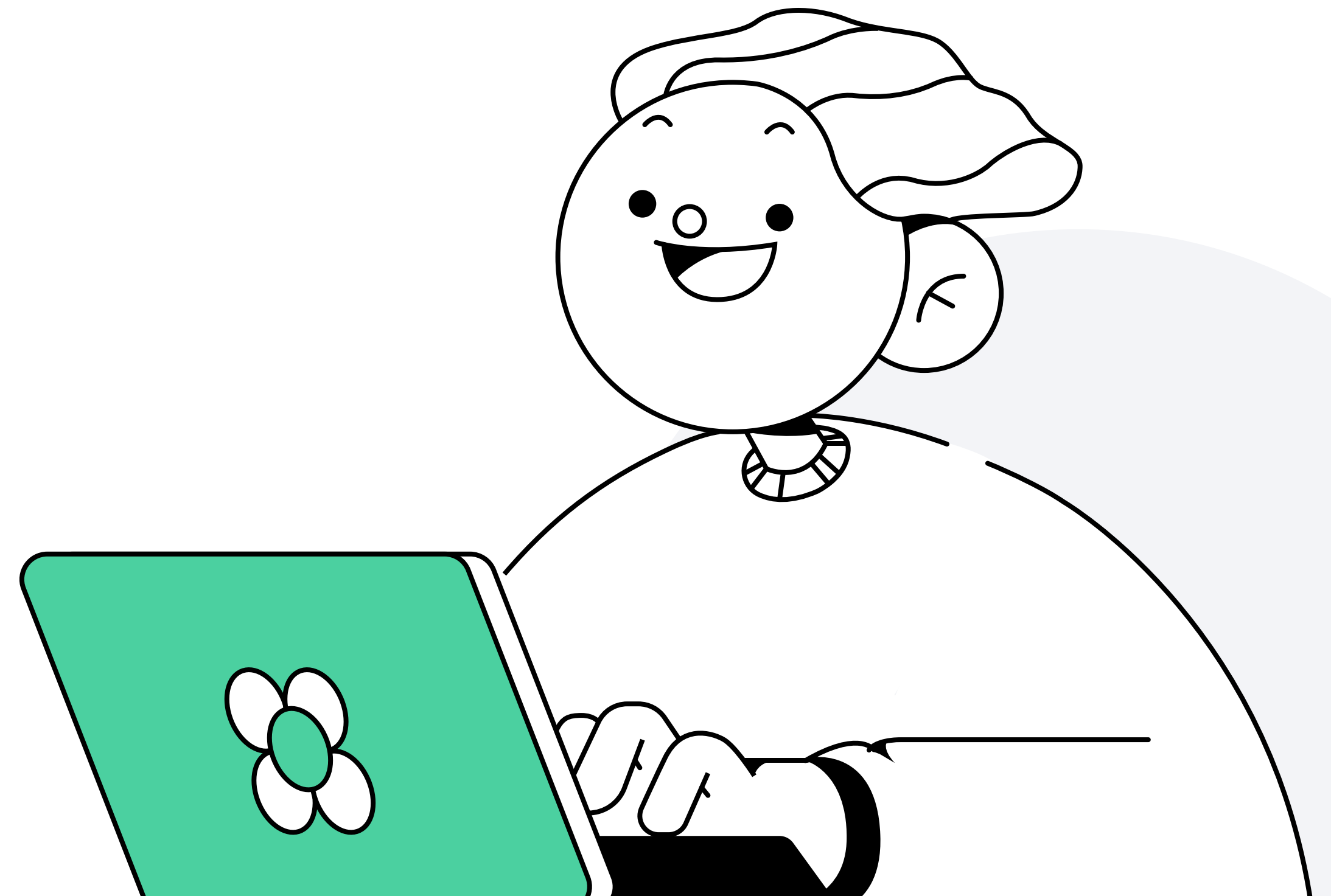


Коллекции List



План занятия

- 1 Класс ArrayList
- 2 Иерархия коллекций
- 3 Методы Collection



Класс ArrayList



Коллекции

Для хранения наборов данных в Java предназначены массивы.

Однако их не всегда удобно использовать, прежде всего потому, что они имеют фиксированную длину.

Эту проблему в Java решают коллекции. Коллекции также предоставляют структуры данных: стек, очередь, дерево и другие

Класс ArrayList

Класс ArrayList — это реализацией списка на основе динамически расширяемого массива. Такую реализацию следует применять, если в процессе работы с коллекцией предполагается частое обращение к элементам по индексу.

ArrayList очень эффективно использует память и операции, основанные на доступе по индексу: сортировка, перемешивание, бинарный поиск и т. д. — выполняются быстро.

Но этой коллекции рекомендуется избегать, если требуется частое удаление/добавление элементов в середину коллекции

Разница между массивом и ArrayList

1

Размер

У массивов в Java фиксированный размер. ArrayList имеет динамически расширяемый размер. При добавлении элементов фактически происходит перераспределение памяти — создание нового массива и копирование в него элементов из старого массива

2

Типы данных

Массив может содержать и примитивы и объекты. Но ArrayList не может содержать в себе данные с примитивным типом. Он может содержать только объекты

3

Iterator

Для перебора элементов в ArrayList используется объект Iterator. Для перебора элементов в массиве используется цикл for

Разница между массивом и ArrayList

4

Безопасность типов

Java обеспечивает безопасность типов, используя Generics. Массив может содержать только объекты одного и того же типа. Если мы попробуем добавить в массив объект производного типа, то получим `ArrayStoreException`

5

Длина

Размер ArrayList может быть получен с помощью метода **size()**. Каждый массив имеет поле **length**, которое означает длину массива

6

Добавление элементов

В ArrayList мы можем использовать метод `add()` для добавления объектов. В массиве добавление элементов невозможно

7

Многомерность

Массив может быть многомерным. ArrayList всегда одномерен

Конструктор и методы List

ArrayList имеет следующие конструкторы:

- **ArrayList()** — создаёт пустой список
- **ArrayList(Collection <? extends E> col)** — создаёт список, в который добавляются все элементы коллекции **col**
- **ArrayList (int capacity)** — создаёт список, который имеет начальную ёмкость **capacity**

Ёмкость в **ArrayList** представляет размер массива, который будет использоваться для хранения объектов. Изначальное задание ёмкости **ArrayList** позволяет снизить перераспределение памяти в процессе расширения, тем самым повышая производительность

Конструктор и методы List

Помимо методов интерфейса Collection интерфейс List имеет собственные методы:

- **get(int index):** возвращает объект из списка по индексу index

```
// получение элемента по индексу  
String p = people.get(3)
```

- **indexOf(Object obj):** возвращает индекс первого вхождения объекта obj в список. Если объект не найден, то возвращается -1

```
// получить индекс первого элемента с указанным значением, если не найдено, то  
-1  
int pos = people.indexOf("Mark");
```

Конструктор и методы List

- **int lastIndexOf(Object obj)** – возвращает индекс последнего вхождения объекта obj в список. Если объект не найден, то возвращается -1

```
// получить индекс последнего элемента с указанным значением, если не найдено то -1
```

```
int pos = people.lastIndexOf("Maria");
```

- **ListIterator listIterator ()** – возвращает объект ListIterator для обхода элементов списка

Конструктор и методы List

- **E set(int index, E obj)** – присваивает значение объекта **obj** элементу, который находится по индексу **index**

```
// обновление значения по индексу  
people.set(1, "Max");
```

- **List subList(int start, int end)** – получает набор элементов, которые находятся в списке между индексами **start** и **end**

```
// получить часть массива  
ArrayList<String> somePeople = new ArrayList<>(people.subList(1, 3));
```


Итерирование по List

Коллекции в Java могут быть разных видов и типов, поэтому встаёт вопрос, как перебрать все элементы коллекции. В этом нам поможет механизм итератора. Рассмотрим на примере, используя разные методы

```
import java.util.Arrays;
import java.util.Iterator;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        List<String> list = new ArrayList();
        list.add("Petya");
        list.add("Olya");
        list.add("Tanya");

        // Обычный метод, который работал на массивах
        for (int i = 0; i < list.size(); i++) {
            System.out.println(list.get(i));
        }
    }
}
```

Итерирование по List

```
// foreach цикл
```

```
for (String s : list) {  
    System.out.println(s);  
}
```

```
// Обычный цикл с итератором
```

```
for (Iterator<String> it = fruits.iterator(); it.hasNext(); ) {  
    String fruit = it.next();  
    System.out.println("fruit: " + fruit);  
}
```

```
// Итератор с циклом while
```

```
Iterator<String> it = list.iterator();  
while (iterator.hasNext()) {  
    String s = iterator.next();  
    System.out.println(s);  
}
```

Итерирование по List

Вывод элементов на экран будет во всех случаях одинаковый, в порядке добавления.

Один из важных методов интерфейса **Collection** — это метод **Iterator iterator()**. Он возвращает итератор — объект, реализующий интерфейс `Iterator`.

Интерфейс `Iterator` имеет следующие сигнатуры методов:

```
public interface Iterator <E>{  
    E next();  
    boolean hasNext();  
    void remove();  
}
```


Итерирование по List

Реализация интерфейса предполагает, что с помощью метода **next()** можно получить следующий элемент.

С помощью метода **hasNext()** можно узнать, есть ли следующий элемент, и не достигнут ли конец коллекции. И если элементы еще имеются, то **hasNext()** вернёт значение true.

Метод **hasNext()** следует вызывать перед методом **next()**, так как при достижении конца коллекции метод **next()** выбрасывает исключение NoSuchElementException.

И метод **remove()** удаляет текущий элемент, который был получен последним вызовом **next()**

Итерирование по List

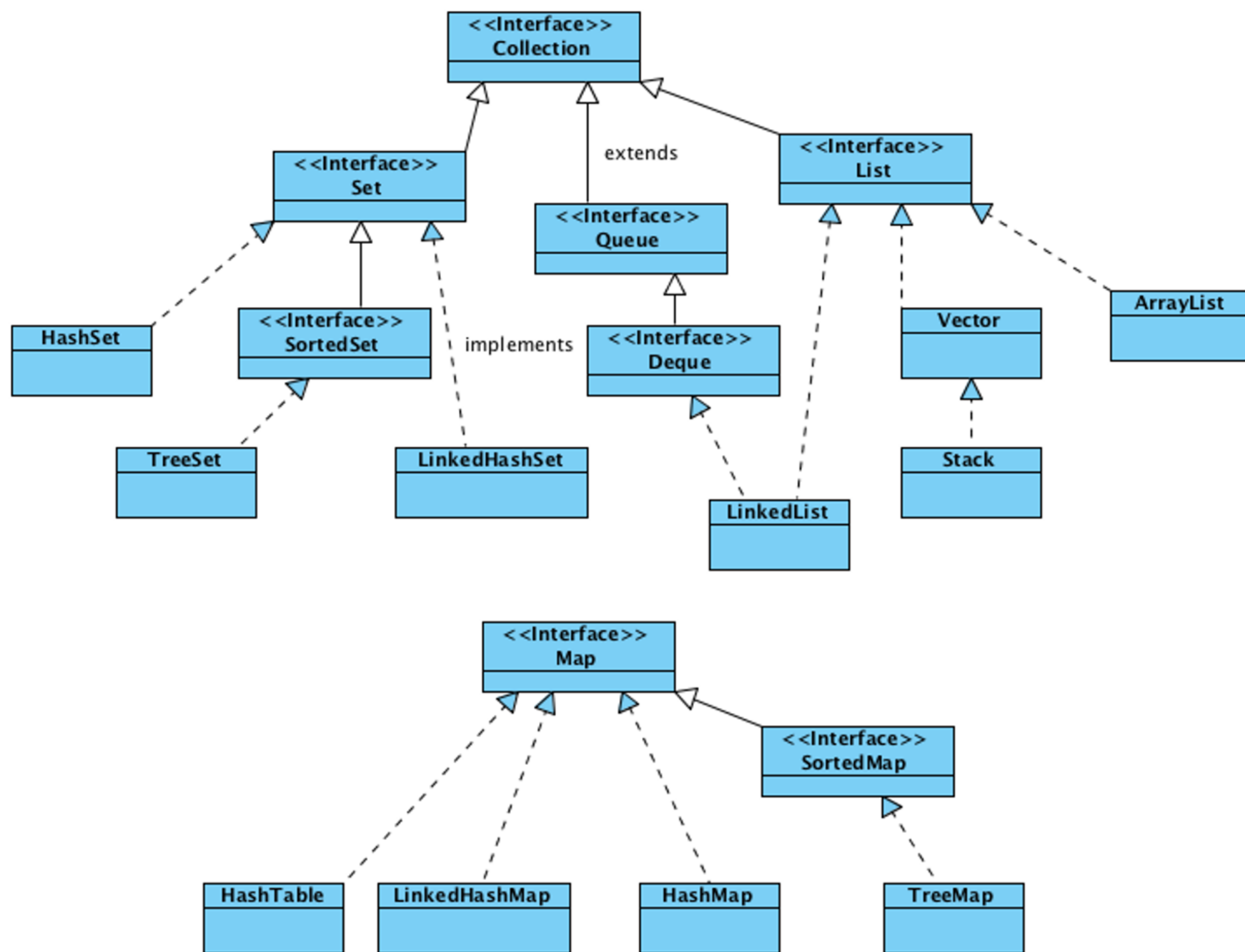
Использовать Iterator нужно в случае, если необходимо в процессе перебора содержимого удалять элементы, иначе, если использовать обычный цикл **for**, получите **ConcurrentModificationException**.

Также его рекомендуется использовать, если необходима уверенность, что не будет произведена попытка доступа к элементу по неправильному индексу и не будет получен **NullPointerException**

Иерархия коллекций

A decorative graphic in the top right corner consisting of several overlapping circles in various shades of teal and green, creating a modern, abstract background element.

Иерархия коллекций



Иерархия коллекций

Главными в иерархии коллекций из пакета **java.util** являются два интерфейса: **Collection** и **Map**. Эти интерфейсы делят все коллекции по типу хранения данных на два вида: простые последовательные наборы элементов и наборы пар «ключ-значение» (словарь или ассоциативный массив)

Интерфейс Collection

Интерфейс Collection предоставляет сигнатуры основных методов работы с простыми последовательными наборами элементов:

- add()
- addAll()
- clear()
- contains()
- containsAll()
- equals()
- hashCode()
- isEmpty()
- iterator()
- remove()
- removeAll()
- size()
- toArray()

Интерфейс Map

Интерфейс Map предоставляет сигнатуры основных методов работы с наборами пар «ключ-значение»:

- clear()
- containsKey()
- containsValue()
- entrySet()
- equals()
- get()
- hashCode()
- isEmpty()
- keySet()
- put()
- putAll()
- remove()
- size()
- values()

Интерфейс Set

Интерфейс Collection расширяют три интерфейса: Set, Queue, List.

Интерфейс Set представляет собой неупорядоченную коллекцию, не содержащую одинаковых элементов. Из-за неупорядоченности элементы множества нельзя получить по их индексу, поэтому для множеств невозможны перемешивание элементов и сортировка.

Однако при этом операции проверки на наличие элемента во множестве эффективнее, чем в списке.

Такая коллекция будет полезна, если необходимо не допускать дубликатов объектов в коллекции и не нужен порядок хранения элементов.

[JavaDoc с описанием методов](#)

Интерфейс Queue

Интерфейс Queue описывает коллекции с предопределённым порядком вставки в очередь и извлечения элементов из очереди.

Эта структура данных используется, например, для хранения списка задач, в приложениях, в которых особенно важно исполнение всех задач из списка и существует определенный порядок исполнения.

Существуют реализации: однонаправленная очередь — FIFO (first-in-first-out), работающая по принципу «первый вошёл — первый вышел», и двунаправленная очередь (интерфейс Deque). Помимо методов интерфейса Collection определяет дополнительные методы для извлечения и добавления элементов в очередь: **offer()**, **peek()**, **poll()**.

[JavaDoc с описанием методов](#)

Интерфейс List

Интерфейс List — один из наиболее часто используемых интерфейсов коллекций, который обозначает упорядоченную коллекцию.

Каждый элемент списка имеет целочисленный индекс, по которому возможно добавление и удаление элементов. Этот интерфейс также предоставляет специальный итератор `ListIterator`, который позволяет перемещаться по списку в обе стороны и вставлять новые элементы.

Такой интерфейс определяет следующие методы: **`get()`**, **`indexOf()`**, **`lastIndexOf()`**, **`listIterator()`**, **`set()`**, **`subList()`**.

[JavaDoc с описанием методов](#)

Интерфейс List

Имплементациями интерфейса List являются следующие классы:

- 1 ArrayList;
- 2 LinkedList;
- 3 Vector.

Рассмотрим их

Класс Vector

Класс Vector очень похож на ArrayList.

Отличия:

- операции работы с Vector — **synchronized**: позволяют только одному потоку в каждый момент времени вносить изменения
- при приближении к максимальному количеству увеличивает размер в 2 раза

В целом считается более устаревшим и используется гораздо реже, чем ArrayList

Класс LinkedList

Класс LinkedList используется гораздо реже.

Является реализацией коллекции на основе двунаправленного связанного списка: каждый элемент имеет ссылку на предыдущий и следующий.

LinkedList эффективнее использовать при вставке/удалении элементов в начале или конце и при вставке/удалении в середине после итерации до нужного места.

Также такая реализация интерфейса List потребляет больше памяти, чем ArrayList

Класс LinkedList

Класс добавляет собственные методы к уже наследованным:

- addFirst();
- addLast();
- getFirst();
- getLast();
- removeFirst();
- removeLast().

Разница в производительности между двумя имплементациями:

Algorithm	array ArrayList	LinkedList
access front	O(1)	O(1)
access back	O(1)	O(1)
access middle	O(1)	O(N)
insert at front	O(N)	O(1)
insert at back	O(1)	O(1)
insert in middle	O(N)	O(1)

Методы Collection



Методы Collection

Среди методов интерфейса Collection можно выделить следующие:

- **boolean add (E item)** — добавляет в коллекцию объект **item**. При удачном добавлении возвращает **true**, при неудачном — **false**

```
ArrayList<String> people = new ArrayList<String>();  
people.add("Tom");  
people.add("Alice");
```

- **boolean addAll (Collection<? extends E> col)** — добавляет в коллекцию все элементы из коллекции **col**. При удачном добавлении возвращает **true**, при неудачном — **false**

```
people.clear();
```

- **void clear()** — удаляет все элементы из коллекции

Методы Collection

- **boolean contains (Object item)** — возвращает **true**, если объект item содержится в коллекции, иначе возвращает **false**

```
boolean isExist = people.contains("Phil")
```

- **boolean isEmpty()** — возвращает **true**, если коллекция пуста, иначе возвращает **false**

```
boolean isEmpty = people.isEmpty()
```

- **Iterator iterator()** — возвращает объект **Iterator** для обхода элементов коллекции

```
Iterator i = people.iterator();
```

```
while(i.hasNext()) {  
    System.out.print(i.next());  
}
```

Методы Collection

- **boolean remove (Object item)** — возвращает **true**, если объект **item** удачно удалён из коллекции, иначе возвращается **false**

```
boolean isRemoved = people.remove("Alex");
```

- **boolean removeAll (Collection<?> col)** — удаляет все объекты коллекции **col** из текущей коллекции. Если текущая коллекция изменилась, возвращает **true**, иначе возвращается **false**

```
people.removeAll(Arrays.asList("Mark", "Maria"));
```

- **boolean retainAll (Collection<?> col)**: удаляет все объекты из текущей коллекции, кроме тех, которые содержатся в коллекции **col**. Если:

```
people.retainAll(Arrays.asList("Tom", "Alice"));
```


Методы Collection

- **int size()** — возвращает число элементов в коллекции

```
int size = people.size();
```

- **Object[] toArray()** — возвращает массив, содержащий все элементы коллекции

```
// получить все элементы списка в виде массива
```

```
String[] peopleArray = people.toArray()
```

Все методы, которые есть в интерфейсе Collection, реализуются всеми коллекциями, поэтому в целом принципы работы с коллекциями будут одни и те же